

Qiskit KG Encodings

Small User Manual

This repository implements a reusable Python pipeline for encoding RDF Knowledge Graphs into quantum states with Qiskit.

It supports three encoding families:

- basis encoding
- amplitude encoding
- phase encoding

The default dataset is included in `data/running_example.ttl`, but the code is designed so you can later replace that file with another Turtle graph and run the same pipeline again.

What The Project Does

The workflow is:

1. load an RDF graph from a file
2. parse the triples
3. build deterministic ID mappings
4. create an encoding-specific representation
5. build a Qiskit circuit
6. simulate locally with Aer
7. save logs and figures

The code is split into reusable modules under `src/`:

- `'kg_parser.py'`: RDF loading and triple extraction
- `'id_mapper.py'`: entity/object and predicate indexing
- `'basis_encoding.py'`: basis-state encoding of triples
- `'amplitude_encoding.py'`: amplitude vectors over triple indices
- `'phase_encoding.py'`: phase-marking rules over triple indices
- `'visualization.py'`: JSON logs and bar-chart figures
- `'main.py'`: command-line entry point

Default Dataset

The included example graph contains six triples:

- Aristotle is typed as Person
- Person is a subclass of Mortal
- Aristotle lives at Athens

- Athens is typed as City
- City is a subclass of Place
- Aristotle teaches Philosophy

This dataset is only a demo target for the thesis repository. The implementation itself is not hardcoded only for these six triples.

Basis Encoding

For basis encoding, each triple (s, p, o) is converted into a computational basis state:

```
subject_bits || predicate_bits || object_bits
```

Important details:

- subject and object share the same ID space
- predicates use a separate ID space
- bit widths are computed dynamically from the loaded graph
- the displayed bitstring is kept explicit so the Qiskit endianness is easier to interpret

The framework can also build a small uniform superposition over the encoded triple states for demonstration purposes.

Amplitude Encoding

Amplitude encoding is defined over triple indices:

- $|i\rangle$ corresponds to the i -th triple in the current graph context
- the vector is padded automatically to the next power of two
- the state is normalized automatically

If no custom weights are provided, the project uses a default strategy such as uniform weights.

For the running example, the experiment script uses the thesis demo vector:

```
[2, 1, 3, 1, 1, 2]
```

which is padded automatically to length 8 before state preparation.

Phase Encoding

Phase encoding is also defined over triple indices.

The pipeline:

1. creates a uniform superposition over the valid triple indices
2. applies a phase shift to marked states
3. applies a Hadamard mixing step to create interference

The marking rule is reusable. It is not hardcoded only for Aristotle.

Examples:

- mark triples by predicate URI
- mark triples by subject URI
- mark triples with a custom condition function

For the included running example, the default demo rule marks triples with predicate `http://example.org/teaches`.

Command-Line Usage

Run the project with:

```
python -m src.main --encoding basis --input data/running_example.ttl
python -m src.main --encoding amplitude --input data/running_example.ttl
python -m src.main --encoding phase --input data/running_example.ttl
```

Useful optional flags:

- '--fixed-thesis-mapping'
- '--weights 2,1,3,1,1,2'
- '--mark-predicate http://example.org/teaches'
- '--mark-subject http://example.org/Aristotle'

Running The Included Experiments

You can also run the thin wrappers in experiments/:

- 'python experiments/exp_basis_running_example.py'
- 'python experiments/exp_amplitude_running_example.py'
- 'python experiments/exp_phase_running_example.py'

These wrappers call the reusable code in `src/` and simply provide default settings for the included thesis example.

Using Another Turtle File

To run the same pipeline on another knowledge graph:

1. prepare another RDF file in Turtle format
2. point the CLI to that file with `--input`
3. optionally pass custom weights or a phase-marking rule

Example:

```
python -m src.main --encoding basis --input path/to/another_graph.ttl
```

The project will automatically rebuild the mappings and required bit widths from the new graph.

Outputs

The pipeline writes outputs under results/:

- 'results/logs/' contains JSON logs
- 'results/figures/' contains plots

The exact outputs depend on the chosen encoding:

- basis: per-triple basis states and an optional superposition summary
- amplitude: vector details and basis-state probabilities
- phase: probabilities before and after the mixing step

The JSON log files also include runtime instrumentation. This means each run records:

- RDF loading and parsing time
- context/mapping construction time
- state preparation time
- statevector simulation time
- measurement simulation time
- plotting time
- log-writing time
- total runtime of the full program

Notes And Limitations

- the current input format is Turtle first
- the code is structured so more RDF serializations can be added later
- the phase oracle uses a simple dense diagonal unitary, which is fine for a small thesis demo but not meant for large-scale graphs
- only local simulation is required; IBM cloud access is not needed

Regenerating This PDF

The source for this document is:

```
docs/manual.md
```

The PDF can be regenerated with:

```
python3 tools/generate_pdf_manual.py
```